



73.64 - Temas Avanzados en Deep Learning

TP2 - Ejercicio 3

Primer Cuatrimestre 2026

Comisión S

Grupo 3

Integrantes:

Nicolás Rossi

Axel Castro Benza

Lucas David Perri

Consideraciones para asegurar Responsible AI y Safety

Gobernanza de datos (Privacidad)

El catálogo se construye con información de fuentes externas como Spotify y Genius (vía Kaggle) y audio descargado, por lo que es necesario revisar licencias, términos de uso y restricciones de redistribución. En particular, las letras son contenido sensible desde la propiedad intelectual: en producción se limitará su exposición, y así evitar mostrar letras sin autorización explícita.

Calidad y representatividad de los datos (Equidad y No Discriminación)

El proyecto contempla canciones populares y menos conocidas, lo que ayuda a reducir el sesgo hacia artistas masivos. Aun así persisten riesgos por idioma, género musical, disponibilidad de letras y calidad del matching entre Spotify y Genius. El pipeline admite canciones sin lyrics mediante prompts "degradados", lo que mantiene la cobertura pero puede producir descripciones menos precisas. Por ello se deberá monitorear el desempeño por subgrupos: canciones con y sin letra, géneros minoritarios y distintos niveles de popularidad, para detectar si quedan sub-representados en las playlists.

Control de alucinación y trazabilidad (Explicabilidad)

El modelo infiere mood, themes y subgénero a partir de metadata y letras, pero estas salidas no deben presentarse como hechos objetivos, sino como inferencias. El sistema debería exponer la razón por-track (qué match semántico y qué atributos motivaron cada canción) e indicar la fuente de la metadata (Spotify / Genius / features de audio), a modo de "cita bibliográfica" que respalde cada decisión. Esto permite explicar resultados y corregir comportamientos no deseados.

Seguridad del contenido y moderación (Robustez y Safety)

Como el sistema procesa letras y consultas abiertas, puede toparse con temas sensibles (violencia, discurso de odio, autolesiones, etc.). Deberían incorporarse mecanismos de moderación de entrada y de salida: filtrar solicitudes maliciosas o inapropiadas, evitar explicaciones ofensivas o discriminatorias y controlar recomendaciones en contextos delicados. Esto complementa las defensas ya existentes del parser, cuyo esquema Pydantic, cerrado por enums, limita el prompt injection. Además de los reintentos y un fallback de cero-resultados de búsqueda que nunca relaja las exclusiones del usuario (p. ej. "nada de reggaetón").

Supervisión humana y mejora continua

Responsible AI requiere mecanismos de supervisión humana y mejora continua. El sistema debería permitir auditorías de playlists generadas, revisión manual de resultados y reindexación ante cambios de prompts, modelos o reglas de negocio. Además, sería deseable incorporar canales de feedback para que los usuarios puedan reportar resultados ofensivos, erróneos o poco relevantes.

Estrategias para poner el modelo en producción

El despliegue en producción de este sistema RAG separa las responsabilidades en dos capas con perfiles operativos fundamentalmente distintos.

La **capa offline** es el pipeline de indexado. Ingesta datos desde las fuentes del proyecto, los datasets de Spotify y Genius, junto con features extraídas mediante Essentia para los tracks no

cubiertos por el endpoint “audio features” de deprecado Spotify, y produce un catálogo enriquecido y consultable. Esta etapa realiza la unificación de registros entre datasets, la limpieza y normalización de las letras, la generación de descripciones semánticas mediante la API de OpenAI, el cálculo de embeddings y la persistencia en PostgreSQL. La capa offline no requiere baja latencia, pero sí tolerancia a fallos, y versionado estricto de artefactos (prompts, identificadores de modelo, revisiones de código). Procesa el catálogo completo a lo largo de varias horas, y los fallos individuales en tracks no deben detener la ejecución.

La **capa offline** está compuesta por operaciones con perfiles de cómputo distintos. La unificación de Spotify + Genius es un join en memoria rápido (minutos). El pipeline de descripción + embedding por LLM es un batch largo dominado por la latencia de la API de OpenAI dentro del bucle, y puede tardar horas. La extracción de features de audio basada en Essentia para tracks desconocidos es cualitativamente diferente: intensiva en CPU (~5-10 segundos por track sobre modelos TensorFlow), con alto uso de filesystem (audio descargado antes de procesarlo), y dependiente de servicios externos sujetos a throttling. Un despliegue correcto trata estas operaciones como jobs separados con asignaciones de recursos y manejo de fallos diferentes: La primera como un script de una sola ejecución, la segunda como un batch con checkpointing y lógica de reintento contra la API, y la tercera como una cola de workers bajo demanda, disparada cuando una consulta de usuario referencia un track fuera del catálogo.

La **capa online** atiende las solicitudes del usuario. Cada request ejecuta una secuencia fija: recibir un prompt en lenguaje natural, invocar un LLM para parsear la intención en un filtro estructurado más una query semántica, ejecutar una query híbrida contra PostgreSQL (filtros SQL sobre columnas de metadata combinados con búsqueda por similitud coseno sobre embeddings vía pgvector), opcionalmente re-ranear, y finalmente invocar un segundo LLM para generar la explicación de la playlist.

La **capa online** requiere sus propias consideraciones operativas, distintas de las de la capa offline: un budget de latencia estricto (objetivo: sub-5 segundos end-to-end), rate limiting por usuario para acotar el costo de LLM, caching de resultados del parseo de intención para prompts comunes, y degradación “elegante” cuando falla la llamada al LLM de generación de explicación (devolver la playlist sin explicación en lugar de fallar el request completo).

Despliegue en la nube. Las dos capas se despliegan de forma distinta. La **capa offline** corre como batch jobs programados o bajo demanda en contenedores, el batch de descripciones LLM de forma semanal o nocturna, y el worker de extracción Essentia sobre una cola disparada por cache misses. Por otro lado, la **capa online** se despliega como una API stateless detrás de un load balancer con autoscaling. Ambas capas comparten la instancia de PostgreSQL.

Consideraciones transversales comunes a ambas capas:

- Almacenamiento de artefactos para el parquet unificado, los archivos de audio descargados y los modelos de TensorFlow (object storage tipo S3)
- Manejo de credenciales para la API key de OpenAI, la URL de la base de datos y futuras keys de otros proveedores (secrets manager)
- Versionado de prompts, y el esquema de base de datos (Alembic para la base de datos, versiones de código para todo lo demás)

A continuación se presenta la arquitectura general propuesta para el pipeline RAG, mostrando la separación entre las capas **offline** y **online**, el rol central de PostgreSQL + pgvector, y los puntos de interacción con las APIs de LLM y embeddings de OpenAI.

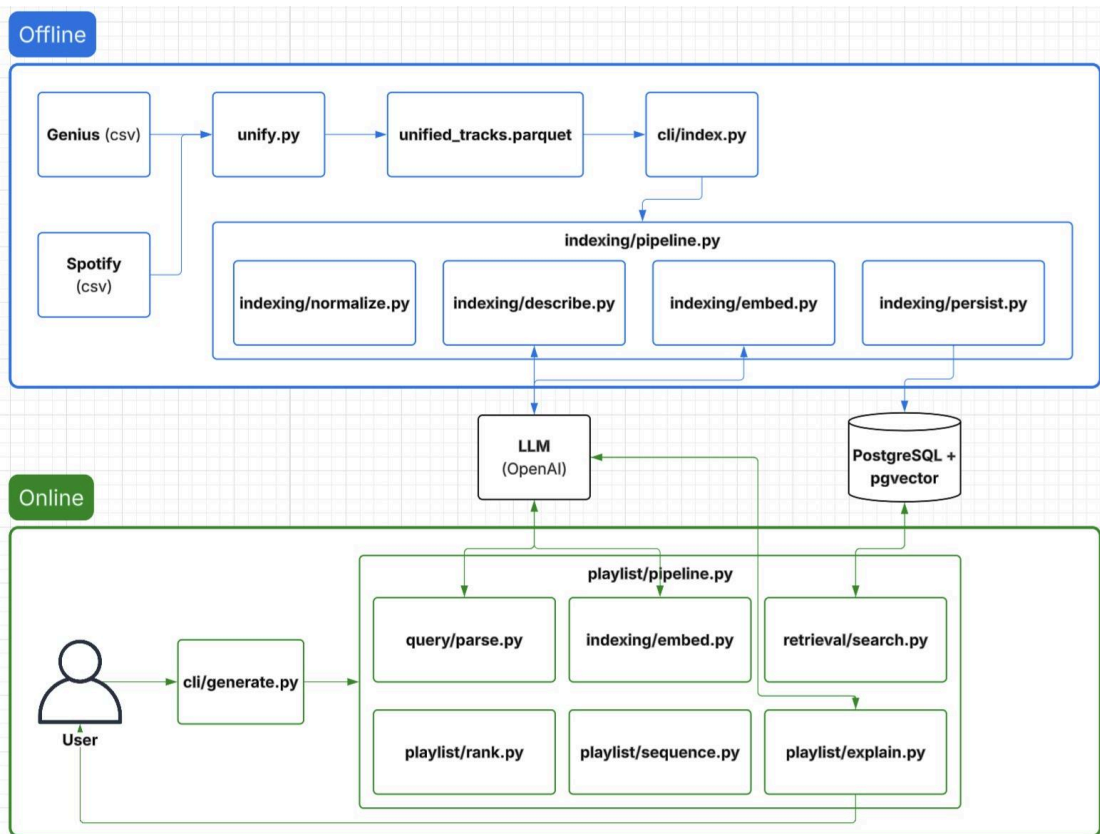


Figura 1: Arquitectura propuesta para el sistema RAG de generación de playlists.